Bu depo aslında sıfırdan yazılmış bir dil değil, büyük ölçüde **GNU Guile 3.0.x'in fork'u** gibi görünüyor. README'de açıkça Guile 3.0 tabanından bahsediliyor ve dizin yapısı da klasik Guile yapısı. [qshell-lan...ware World]

Eğer amacın:

- **QShell Language** adında Guile türevi bir dil yapmak,
- ve bunu **UMFS** projesinde Bash yerine kullanmaksa,

özellikle şu bölgelere dikkat etmeni öneririm:

1. Dilin çekirdeği: `libguile/`

Burası yorumlayıcının ve VM'nin kalbi.

Önemli sebepler:

- Yeni sözdizimi (syntax) eklemek
- Yeni primitive'ler eklemek
- Shell odaklı veri tipleri oluşturmak
- Process, pipe, fd yönetimi eklemek

için en kritik klasör budur. [qshell-lan...ware World]

---

2. Scheme standartlarını bozmamaya çalış

Bence en büyük hata:

Guile'ı tamamen kırıp yeni bir dil yapmak

olur.

Daha mantıklı yol:

(use-modules (qshell process))
(use-modules (qshell fs))

şeklinde gitmek.

Yani:

- Scheme çekirdeği aynı kalsın
- Shell özellikleri modül olarak gelsin

Bu sayede:

(ls "/tmp")
(pipe

(grep "error")
(sort))

gibi yapılar eklenebilir.

---

3. `module/` klasörünü incele

Burada Guile'ın standart kütüphaneleri var. [qshell-lan...ware World]

QShell modüllerini burada konumlandırabilirsin:

module/qshell/

module/qshell/fs.scm
module/qshell/process.scm
module/qshell/network.scm
module/qshell/package.scm

---

4. Shell için FFI gücünü kullan

Guile'ın önemli avantajlarından biri:

- libffi
- foreign interface

desteğinin olması. [qshell-lan...ware World]

Bu sayede:

(openat ...)
(epoll ...)
(io_uring ...)
(clone ...)

gibi Linux sistem çağrılarına ulaşabilirsin.

Bash'in yapamadığı birçok şeyi yaparsın.

---

5. Bash yerine geçecekse process modeli önemli

Asıl kritik nokta syntax değil.

Bash'i güçlü yapan:

cat a.txt | grep foo | sort

ifadesidir.

QShell'de buna karşılık:

```
(pipe
 (cat "a.txt")
 (grep "foo")
 (sort))
```

veya

```
(-> (cat "a.txt")
   (grep "foo")
   (sort))
```

gibi DSL tasarlayabilirsin.

Bence başarının %80'i burada.

---

6. Yeni parser yazmak zorunda değilsin

Ben olsam:

Scheme sözdizimini korurum.

Çünkü:

```
(install "gcc")
(sync system)
(update packages)
```

şeklindeki ifadeler zaten okunabilir.

Parser çatallanırsa Guile ile uyumluluk azalır.

---

7. UMFS açısından avantajları

UMFS'te Bash yerine QShell kullanırsan:

- ✅ Tip güvenliği
- ✅ Daha güçlü hata yönetimi
- ✅ Fonksiyonel programlama
- ✅ Modüler paket sistemi
- ✅ Gömülü yorumlayıcı

✅ C kütüphanelerine doğrudan erişim

elde edersin.

Özellikle

(package-install "nginx")
(service-enable "nginx")

tarzı dağıtım yönetimi DSL'leri yazmak çok kolaylaşır.

Benim dikkat edeceğim dosya ve dizinler

İnceleme sırası:

libguile/
module/
ice-9/
scripts/
guild/

özellikle:

libguile/
module/

ilk okunacak yerler olur. [qshell-lan...ware World]

Mimari olarak ise QShell'i "yeni bir Scheme" değil:

> *"Guile tabanlı, Bash'in yerini alacak sistem programlama ve otomasyon dili"*

olarak konumlandırman çok daha mantıklı olur. Böylece hem GNU Guile ekosisteminden kopmazsın hem de UMFS'e özgü güçlü bir shell dili elde edersin.